

Atividade Prática 02 e no arquivo `brilcalc.log` (luminosidade integrada, Run2023D)

August 17, 2026

Modalidade: individual, entrega via link do repositório no GitHub

Objetivos de aprendizagem

Ao final desta atividade, você deve ser capaz de:

1. Compilar um programa C++ simples e automatizar a compilação com `make`.
2. Escrever *job scripts* que disparam vários processos em paralelo, tanto em Bash quanto em Python (usando `multiprocessing`, como visto no slide “Multiprocessamento”).
3. Usar expressões regulares (`re`) em Python para extrair informação de um arquivo de texto estruturado.
4. Praticar novamente o fluxo de `git` (`fetch/merge`) para obter arquivos de um repositório do professor.

Contexto

Você vai resolver dois exercícios independentes, ambos em um novo repositório chamado `python`. Crie o repositório no GitHub e clone-o localmente antes de começar (mesmo procedimento da atividade anterior).

```
mkdir python && cd python
git init
git branch -M main
git remote add origin <URL-do-seu-repositorio>
```

Organize as duas partes em subpastas `Ex1/` e `Ex2/` dentro do repositório.

1 Exercício 1 — C++, Make e execução paralela

1a) Programa C++

Dentro de `Ex1/`, escreva um programa `hello.cpp` que:

- recebe um número como parâmetro de linha de comando;
- escreve em `stdout` a mensagem `"Hello world <número>"`.

Exemplo de uso esperado:

```
$ ./hello 7
Hello world 7
```

Dica: os argumentos de linha de comando chegam em `argv` (análogo ao `sys.argv` do Python, visto no slide “Parâmetros de Linha de Comando”); use `std::atoi(argv[1])` para converter o argumento em inteiro.

1b) Makefile

Escreva um Makefile (dentro de Ex1/) que compile e linke o programa, com pelo menos os targets `all` e `clean`:

```
make          # gera o executavel "hello"
make clean    # remove o executavel
```

1c) Job script em Bash

Escreva um script `run_jobs.sh` que roda $n = 10$ jobs **em paralelo**, cada um chamando `./hello` com um número de entrada diferente (1 a 10), salvando a saída de **cada job em um arquivo separado** (ex: `output_1.txt`, ..., `output_10.txt`).

Dica: no Bash, colocar `&` ao final de um comando o manda rodar em segundo plano (*background*); o comando `wait` espera todos os processos em segundo plano terminarem antes de continuar — é assim que se consegue paralelismo simples em um script shell (o mesmo princípio de redirecionamento visto no slide “Redirecionamento e Pipes” da Semana 2, aplicado a vários processos de uma vez).

1d) Job script em Python

Escreva um script equivalente `run_jobs.py`, usando o módulo `multiprocessing` (como no slide “Multiprocessamento”), que também roda os 10 jobs em paralelo chamando o executável `./hello` (via `subprocess`) e grava a saída de cada um em um arquivo separado.

Checkpoint: depois de rodar *qualquer um* dos dois scripts, devem existir 10 arquivos `output_1.txt` a `output_10.txt` em Ex1/, cada um contendo `"Hello world <n>"` com o `n` correspondente.

2 Exercício 2 — Luminosidade integrada com re

No CMS, a luminosidade integrada dos dados de colisão é calculada com um programa chamado `bril`. No repositório do professor, na subpasta **Exercícios**, está o arquivo de saída do `bril` (`brilcalc.log`) de um conjunto de dados do Run2023D (tomada de dados de 2023).

2a) Obtendo o arquivo via merge

Ao contrário da atividade anterior (onde você criou tudo do zero), aqui você vai **buscar um arquivo pronto do repositório do professor** e mesclá-lo ao seu próprio repositório — praticando exatamente o fluxo do slide “Git: Fetch e Push”:

```
# Adicione o repositorio do professor como um remoto extra
git remote add professor https://github.com/dune-mdias/F056/

# Busque os dados desse remoto
git fetch professor
```

```
# Mescle a branch indicada pelo professor (ajuste o nome se necessario)
git merge professor/main --allow-unrelated-histories
```

Depois do merge, você deve ter o arquivo `Ex2/brilcalc.log` disponível localmente. Se o merge gerar conflitos, resolva-os como na atividade anterior (Parte 4 da atividade de Git/Make).

2b) Formato do arquivo

O arquivo tem uma tabela detalhada por `run/fill` e, ao final, um bloco `#Summary:` com uma única linha resumindo o total, por exemplo:

```
#Summary:
+-----+-----+-----+-----+-----+-----+
| nfill | nrun | nls   | ncms  | totdelivered(/pb) | totrecorded(/pb) |
+-----+-----+-----+-----+-----+-----+
| 21    | 33    | 23628 | 23628 | 8240.552781490    | 7853.687813676    |
+-----+-----+-----+-----+-----+-----+
```

2c) O script

Escreva um script `luminosidade.py` (dentro de `Ex2/`) que:

1. leia `brilcalc.log`;
2. use uma expressão regular (módulo `re`, como visto nos slides “Expressões Regulares”) para extrair o valor da coluna `totrecorded(/pb)` da linha de dados do bloco `#Summary:` — **não** vale procurar a string manualmente com `split()` sem usar `re`;
3. converta o valor de pb^{-1} para fb^{-1} (lembre-se: $1 \text{ fb} = 1000 \text{ pb}$);
4. imprima o resultado na tela, **com 1 casa decimal**, no formato:

```
Luminosidade integrada recorded: 7.9 fb^-1
```

Dica: repare que a linha do `#Summary:` tem 6 números separados por `|`; um padrão como `r"\|s*(\d+)\s*\|..."` combinando grupos numéricos, um por coluna, com `re.search()`, resolve o problema de forma robusta — assim como no exemplo do slide “RE - Exemplo com Dados”, que extrai `dataVersion` de uma linha de configuração usando grupos nomeados.

Checkpoint: rodando `python3 luminosidade.py` a partir de `Exercicios/`, com o `brilcalc.log` fornecido, a saída deve ser exatamente `7.9 fb^-1` (valor já conferido pelo professor a partir do arquivo real).

Entrega

Envie o link do seu repositório no GitHub. Verifique antes de entregar que:

- ☐ O repositório é público (ou o professor foi adicionado como colaborador).
- ☐ `Ex1/` contém `hello.cpp`, `Makefile`, `run_jobs.sh` e `run_jobs.py`, todos funcionais.
- ☐ `Ex2/` contém `brilcalc.log` (obtido via merge) e `luminosidade.py`, funcional.
- ☐ `git log -oneline` mostra commits organizados por etapa (não um único commit gigante).
- ☐ A partir de um clone limpo, os comandos abaixo funcionam sem intervenção manual:

```
cd Ex1 && make clean && make all && ./run_jobs.sh  
cd ../Ex2 && python3 luminosidade.py
```

Dica final

Se o `make` reclamar de “missing separator”, lembre-se: os comandos dentro de cada regra do `Makefile` precisam começar com **TAB**, não espaços (mesmo erro clássico da atividade anterior). E se o `git merge` do Exercício 2 reclamar de históricos não relacionados, use a flag `-allow-unrelated-histories`, como mostrado no comando acima.